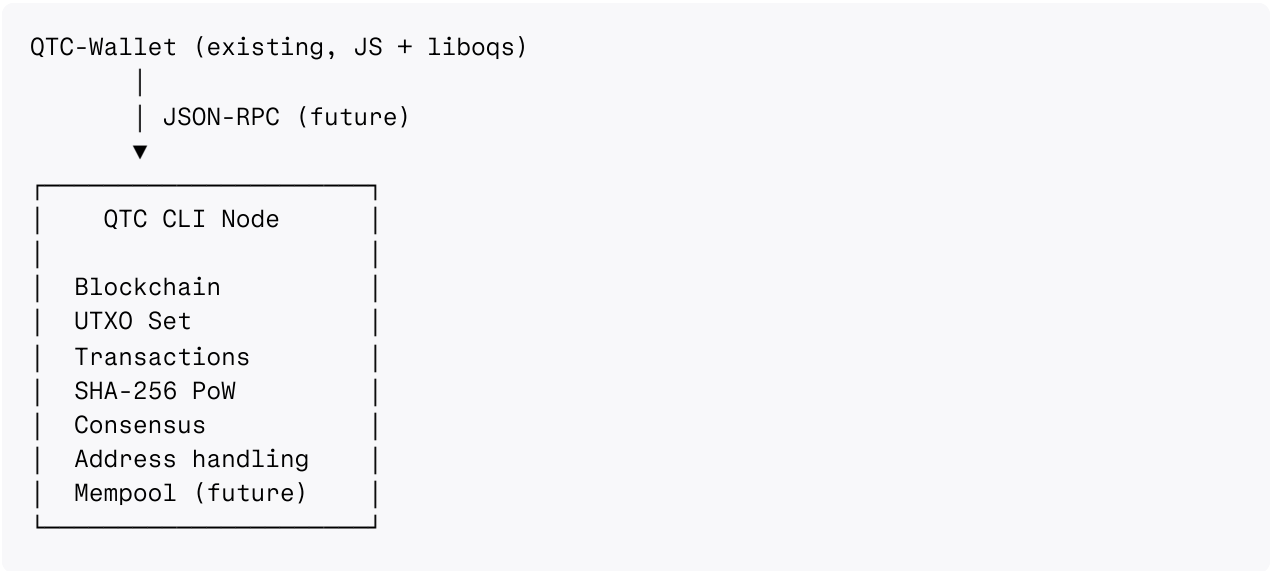


QTC Protocol Development Plan

Executive Summary

Build a small standalone QTC Protocol CLI in C++ to prove the blockchain protocol works end-to-end (blocks, transactions, UTXOs, PQ signatures, addresses, mining) before porting into BitcoinII-Core. The existing QTC-Wallet stays as the reference wallet and test-vector producer. PQNativeConcept becomes the research archive.

Architecture



v1 Protocol Parameters (Frozen)

Parameter	Value
Max supply	21,000,000 QTC
Block interval	~10 minutes (600s)
Max block size	32 MB (development target)
PoW algorithm	SHA-256d (double SHA-256)
PoW algorithm ID	0x01
Signature algorithm	ML-DSA-65 (Dilithium3 / FIPS-204)
Signature algorithm ID	0x01
KEM algorithm	ML-KEM-1024 (Kyber1024 / FIPS-203)
KEM algorithm ID	0x01
Address format	bech32m, HRP qtc (mainnet) / tqtc (testnet)

Parameter	Value
Witness version	1 (standard PQ wallet)
Coinbase maturity	100 blocks
Initial subsidy	50 QTC
Halving interval	210,000 blocks

Three Frozen Interfaces

1. Transaction Format (QTC TX v1)

```

Transaction
├─ version (4 bytes, LE) = 1
├─ inputs[]
│   ├── previous_txid (32 bytes)
│   ├── previous_vout (4 bytes, LE)
│   ├── sequence (4 bytes, LE)
│   └─ unlock/witness data
├─ outputs[]
│   ├── amount (8 bytes, LE, in satoshis)
│   └─ script/address commitment
├─ locktime (4 bytes, LE)
└─ PQ witness (appended)
    ├── algorithm_id (1 byte)
    ├── public_key (ML-DSA-65, 1952 bytes)
    └─ signature (ML-DSA-65, up to 3293 bytes)

```

2. Address Format (qtc1)

Dilithium3 public key → SHA3-256 → first 20 bytes → bech32m → qtc1...

Mainnet HRP: qtc

Testnet HRP: tqtc

3. Wallet ↔ Node RPC (JSON-RPC)

```

getblockchaininfo
getblock
getblockhash
gettransaction
getrawtransaction
getnewaddress
validateaddress
getbalance
listunspent
createrawtransaction
signrawtransaction
sendrawtransaction

```

```
getmempoolinfo  
getrawmempool  
getblocktemplate  
submitblock
```

Sprint 1 — Days 1-7 (Foundation)

Day 1: Project Structure + Consensus Constants ✓

- Clean C++17 project with CMake + Makefile
- Headers: consensus.h, crypto.h, transaction.h, block.h, chain.h, pq_crypto.h
- Consensus constants: 21M supply, 10-min blocks, 32MB target, SHA-256 PoW ID
- Version IDs for all subsystems (block, tx, PoW, sig, KEM, witness, address)
- PQ crypto placeholder interface (no liboqs dependency yet)

Day 2: Block + Genesis + SHA-256 ✓

- Self-contained SHA-256 implementation (FIPS 180-4)
- Double SHA-256 (Bitcoin-style hashing)
- Block header serialization (80 bytes, Bitcoin-compatible)
- Merkle root computation
- Genesis block creation (deterministic, fixed timestamp)
- Chain state management
- CLI commands: init, getblockchaininfo, getblock, getblockhash, validatechain, selftest
- 15/15 self-tests passing

Day 3: Transactions + UTXO Set (NEXT)

- Transaction serialization/deserialization
- UTXO database (in-memory map: txid+vout → UTXO)
- Balance calculation from UTXO set
- Manual transaction creation (Alice → Bob)
- UTXO spending and creation
- Coinbase transaction support

Day 4: SHA-256 Mining

- CPU miner: nonce increment → double_sha256(header) → target comparison
- Regtest difficulty: trivially easy
- `qtc mine [n] [address]` command
- Block template creation

- Chain validation (PoW, linkage, merkle root)

Day 5: PQ Wallet Integration

- Link liboqs (or use existing oqs_wallet_cli as subprocess)
- Implement ML-DSA-65 key generation
- Implement ML-DSA-65 sign/verify
- Freeze sighash construction (what bytes get signed)
- Freeze public key encoding
- Freeze signature encoding
- Create permanent test vectors from QTC-Wallet
- Address compatibility: QTC-Wallet generates qtc1... → QTC Node decodes it

Day 6: RPC + Send/Receive

- Implement JSON-RPC server (HTTP)
- End-to-end flow: mine → UTXO → wallet signs → node validates → mempool → mine → confirm
- `qtc-cli mine 101 alice` → 50 QTC
- `qtc-cli send bob 10` → signed transaction
- `qtc-cli mine 1` → confirmed
- `qtc-cli getbalance bob` → 10 QTC

Day 7: Integration Tests

- Bad signature → REJECTED
- Double spend → REJECTED
- Fake amount → REJECTED
- Invalid block → REJECTED
- Bad PoW → REJECTED
- Node restart → state preserved
- Chain persistence

Sprint 2 — Mining → Receive → Send → Validate

- Mempool implementation
- Block template with pending transactions
- Mining with transaction inclusion
- Confirmation logic

Sprint 3 — Two-Node P2P

- Node A (127.0.0.1:18444) ↔ Node B (127.0.0.1:18445)
- Block propagation
- Transaction relay
- Chain synchronization

Sprint 4 — QTC-Wallet Compatibility

- Replace internal wallet with actual QTC-Wallet
- Test: QTC-Wallet generates address → QTC Node recognizes it
- Test: QTC-Wallet signs → QTC Node verifies with ML-DSA
- Test: QTC Node creates UTXO → QTC-Wallet sees it

Sprint 5+ — BitcoinII Integration

1. Port QTC monetary policy
2. Port QTC genesis
3. Port SHA-256 PoW
4. Port qtc1 address format
5. Port ML-DSA transaction type
6. Port wallet interface
7. Port ML-KEM (where required)
8. Port PQ P2P enhancements
9. Future: qRandomX mining research

What Is NOT Being Built Yet

Deferred to `future/` directory:

- qRandomX / RandomX / Cuckoo mining
- PQ Noise P2P transport
- Witness compression
- OP_CAT and complex script extensions
- Hardware wallet integration
- Block explorer / GUI
- Advanced mining tools

Repository Strategy

Repository	Role
QTC-Protocol (this project)	Protocol laboratory — prove the protocol
QTC-Core (future)	BitcoinII-derived production node
QTC-Wallet (existing)	Reference wallet + test vectors
PQNativeConcept (existing)	Research archive

Build Instructions

```
# Using Makefile (no cmake required)
make
./qtc init
./qtc selftest

# Using CMake (if available)
cmake -S . -B build
cmake --build build
./build/qtc init
./build/qtc selftest
```

Key Design Principles

1. **Write → Build → Test → Commit** for every component
2. **No external dependencies** for Days 1-2 (SHA-256 is self-contained)
3. **Fixed genesis** — deterministic timestamp for reproducible hash
4. **Version everything** — block, tx, PoW, sig, KEM, witness, address all have version IDs
5. **UTXO model** — no account balances, wallet computes from UTXO set
6. **CLI first, RPC second** — JSON-RPC wraps the same core methods
7. **PQ crypto is modular** — pq_crypto.h interface exists now, liboqs links on Day 5